

Production Scheduling under Resource and Setup-Cost Constraints: A Mixed-Integer Programming Approach

Ali Akarsu

1 Introduction

Production planning is a classic operations-research decision problem: a manufacturer must decide how much of each product to make so as to maximise profit while respecting limited resources (Winston 2004). When some decisions are discrete (whole units of product) and others are logical switches (whether to incur a setup cost), the problem becomes a *Mixed-Integer Program* (MIP) rather than a continuous Linear Program (LP) (Taha 2017).

This report formulates and solves a five-product factory scheduling problem as a MIP. The factory makes products P1–P5, each consuming labour, machine time and raw materials and yielding a fixed unit profit. Production is bounded by resource availability and side constraints, and producing P5 triggers a fixed setup cost. The problem is treated as a combined constraint-satisfaction and optimisation task (Russell & Norvig 2010): constraints define the feasible region, the objective selects the best feasible point. It is implemented in Python with the PuLP library and the CBC branch-and-bound solver (Mitchell et al. 2011, Forrest & Lougee-Heimer 2005), and the MIP solution is compared with its LP relaxation to quantify the cost of integrality.

2 Problem Formulation

2.1 Decision Variables

Let x_p denote the number of units of product $p \in \{P1, \dots, P5\}$ produced. Because units are indivisible, each x_p is a non-negative integer. Product P5 additionally requires a binary indicator y_5 that equals 1 when any P5 is produced and 0 otherwise; this variable carries the fixed setup cost and turns the model into a *fixed-charge* problem (Williams 2013):

$$x_p \in \mathbb{Z}_{\geq 0}, \quad p \in \{P1, \dots, P5\}, \quad y_5 \in \{0, 1\}. \quad (1)$$

2.2 Objective Function

The objective maximises total profit net of the P5 setup cost:

$$\max Z = 5x_1 + 4x_2 + 6x_3 + 7x_4 + 8x_5 - 30y_5. \quad (2)$$

2.3 Constraints

The three resources impose capacity constraints:

$$2x_1 + 3x_2 + 1x_3 + 4x_4 + 5x_5 \leq 95 \quad (\text{labour}) \quad (3)$$

$$3x_1 + 2x_2 + 4x_3 + 1x_4 + 3x_5 \leq 80 \quad (\text{machine time}) \quad (4)$$

$$4x_1 + 1x_2 + 3x_3 + 2x_4 + 2x_5 \leq 70 \quad (\text{raw materials}) \quad (5)$$

The side constraints encode the additional production rules:

$$x_1 \leq 20 \quad (\text{P1 capacity}) \quad (6)$$

$$x_2 + x_3 \geq 15 \quad (\text{combined P2+P3 minimum}) \quad (7)$$

$$x_4 \geq 5 \quad (\text{P4 minimum}) \quad (8)$$

Finally, the fixed-charge linking constraint connects production of P5 to the binary setup switch:

$$x_5 \leq M y_5, \quad M = \lfloor \frac{95}{5} \rfloor = 19. \quad (9)$$

If $x_5 > 0$ then y_5 must equal 1, so the \$30 cost is paid; if $y_5 = 0$ then x_5 is forced to 0. The constant M is set to the tightest valid upper bound on x_5 implied by the labour constraint (since $5x_5 \leq 95$), because a smaller M yields a tighter LP relaxation and therefore a more efficient branch-and-bound search (Williams 2013). The reverse direction ($y_5 = 1, x_5 = 0$) need not be excluded explicitly: paying the setup cost without producing P5 can never be optimal, so the solver naturally sets $y_5 = 0$ in that case.

3 Implementation

The model was implemented in Python 3 using PuLP, an algebraic modelling layer that dispatches the problem to the bundled CBC solver (Mitchell et al. 2011, Forrest & Lougee-Heimer 2005). CBC solves the MIP exactly with LP-based branch-and-bound: it solves LP relaxations, branches on fractional integer variables, and prunes nodes whose bound cannot beat the incumbent (Taha 2017).

A single builder function constructs both models; only the variable category differs. For the MIP, x_p are `Integer` and y_5 is `Binary`; for the LP relaxation all integrality is dropped (x_p `Continuous`, $y_5 \in [0, 1]$). Listing 1 shows the core of the formulation, which maps line-for-line onto the equations above. The full documented script is provided as a separate deliverable.

Listing 1: Core of the PuLP formulation (objective, constraints, setup link).

```

1 prob += pulp.lpSum(PROFIT[p]*x[p] for p in PRODUCTS) - 30*y5 # objective
2 prob += pulp.lpSum(LABOUR[p]*x[p] for p in PRODUCTS) <= 95 # labour
3 prob += pulp.lpSum(MACHINE[p]*x[p] for p in PRODUCTS) <= 80 # machine
4 prob += pulp.lpSum(RAW[p]*x[p] for p in PRODUCTS) <= 70 # raw
5 prob += x["P1"] <= 20 # P1 capacity
6 prob += x["P2"] + x["P3"] >= 15 # P2+P3 minimum
7 prob += x["P4"] >= 5 # P4 minimum
8 prob += x["P5"] <= 19*y5 # fixed-charge link
9 prob.solve(pulp.PULP_CBC_CMD(msg=0))

```

4 Results and Analysis

4.1 Optimal Solutions

Both models solved to proven optimality. Table 1 reports the solutions and Figure 1 visualises the production plan and resource usage.

Table 1: LP relaxation versus integer (MIP) solution.

Model	x_1	x_2	x_3	x_4	x_5	y_5	Objective
LP relaxation	0	5	10	17.5	0	0	\$202.50
MIP (integer)	0	7	10	16	0	0	\$200.00

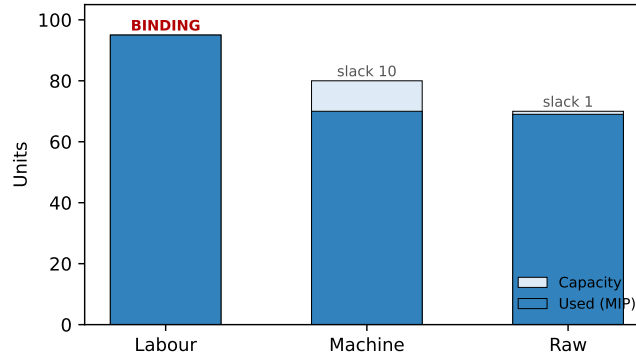


Figure 1: Resource utilisation at the MIP optimum. Labour is fully consumed (binding); machine time and raw materials retain slack.

4.2 LP Relaxation versus MIP

The LP relaxation returns \$202.50 and the MIP \$200.00, a gap of \$2.50 (1.25%). Removing integrality enlarges the feasible region, so the LP optimum is always an *upper bound* on the integer optimum (Taha 2017). The gap cannot simply be rounded away: rounding the LP solution down (P4 from 17.5 to 17) gives a feasible plan worth only \$199, strictly worse than the integer optimum of \$200. The MIP instead rebalances the mix — raising P2 to 7 and lowering P4 to 16 — recovering the lost profit. Integer problems therefore need dedicated solving, not naive rounding (Williams 2013).

4.3 Interpretation and the Setup Cost

Labour is the single binding resource: all 95 units are consumed, while 10 units of machine time and 1 of raw materials remain unused (Figure 1). Labour is therefore the bottleneck, and the best products are those yielding the most profit per unit of labour — which is why P3 (profit/labour = 6.0) is pushed to its machine/raw limits and P4 is produced well above its minimum of 5.

Notably, P5 is *not* produced ($x_5 = y_5 = 0$) despite its highest unit profit (\$8): it is the most labour-intensive product (5 units each, on the binding resource) and incurs the \$30 setup cost. Forcing at least one unit of P5 drops the objective from \$200 to \$173, confirming the fixed-charge mechanism works and that the setup is uneconomical here. The model thus acts as a constraint-satisfaction-plus-optimisation system (Russell & Norvig 2010): the link (9) encodes the setup logic and the optimiser rationally leaves the switch off. In practice this tells the manager to specialise in P2, P3 and P4, to expand *labour* if adding capacity, and to avoid the P5 line under the current costs.

5 Limitations and Future Work

The formulation is correct but rests on simplifying assumptions. The data are *deterministic and static*, whereas real demand and prices fluctuate. The model is *single-period*, so it cannot hold inventory or amortise the P5 setup over future output. Profit is *linear*, ignoring economies of scale, and a setup cost is modelled only for P5, making the products asymmetric. The big- M link, though given a tight M , can still weaken the LP relaxation on larger instances (Williams 2013).

Natural extensions include a *multi-period* model with inventory to spread setup costs; *setup switches for every product*; a *sensitivity analysis* on the binding labour constraint (its shadow price) to value extra capacity; and a *stochastic or robust* reformulation for uncertain data.

6 Conclusion

This report modelled a five-product factory schedule as a mixed-integer program with a fixed-charge setup cost, solved it in PuLP/CBC, and compared the integer solution with its LP relaxation. The optimal plan earns \$200 (P2 = 7, P3 = 10, P4 = 16; P1, P5 excluded). Labour is the sole bottleneck, and the setup cost correctly deters the high-profit P5. The 1.25% LP–MIP gap, and the sub-optimality of rounding, confirm the value of solving such problems as true integer programs.

References

- Forrest, J. & Lougee-Heimer, R. (2005), CBC user guide, *in* ‘Emerging Theory, Methods, and Applications (INFORMS TutORials in Operations Research)’, INFORMS, pp. 257–277.
- Mitchell, S., O’Sullivan, M. & Dunning, I. (2011), PuLP: A linear programming toolkit for Python, Technical report, The University of Auckland, Auckland, New Zealand.
- Russell, S. J. & Norvig, P. (2010), *Artificial Intelligence: A Modern Approach*, 3rd edn, Pearson, Upper Saddle River, NJ.
- Taha, H. A. (2017), *Operations Research: An Introduction*, 10th edn, Pearson, Harlow, UK.
- Williams, H. P. (2013), *Model Building in Mathematical Programming*, 5th edn, Wiley, Chichester, UK.
- Winston, W. L. (2004), *Operations Research: Applications and Algorithms*, 4th edn, Brooks/Cole, Cengage Learning, Belmont, CA.